Credit Card fraud detection AzureMl

Jyotheswar Narayana Narravula
Narravula.4@wright.edu

Mohith Reddy Kovvuri
Kovvuri.6@wright.edu

Vaishnavi Madduri
Madduri.10@wright.edu

Abstract
Credit card fraud detection is an important and challenging problem in the financial domain because fraudulent transactions are rare and are often hidden within a very large number of legitimate transactions. In this project, we used the Kaggle Credit Card Fraud Detection dataset, which contains 284,807 transactions and exhibits a highly imbalanced class distribution, with only a small fraction belonging to the fraud class. To address this problem, we developed a structured fraud detection workflow using Microsoft Azure Machine Learning and Python. Azure Machine Learning was used for dataset registration, experiment setup, and automated model training through AutoML, while Python-based libraries were used for exploration data analysis and visualization. As part of the analysis, we examined important dataset characteristics such as class imbalance, transaction amount distribution, feature relationships, and time-based fraud patterns. The project was designed to study how cloud-based machine learning tools and data analysis techniques can be combined in a practical way to support fraud detection on highly imbalanced financial transaction data.

Keywords: Credit card Fraud detection, Azure Machine Learning, AutoML, Random Forest, XGBoost, VotingEnsemble, class imbalance, credit card transactions

GitHublink:https://github.com/jyotheswarnarayana/credit-card-fraud-detection-azureml

I. INTRODUCTION

Fraud detection is important for banks and other financial companies. They must deal with a number of transactions every day. It is very hard to find the ones just by looking at them manually. Fraudulent transactions are not very common. Are usually hidden among a lot of legitimate ones. This makes it very difficult to detect them.

The main problem we are trying to solve is to find credit card transactions. The dataset we are using has a lot of normal transactions than fraudulent ones. This creates a challenge for machine learning models because they can still be very accurate overall but fail to identify the cases. So we need to build a system that can learn from these events and provide useful insights into patterns related to fraud.

This problem is important because fraud can have an impact beyond just financial loss. It can make customers lose trust hurt the reputation of companies and increase the costs of investigating and preventing fraud. A good fraud detection system can help identify behavior early reduce manual work and make financial systems more secure. This system should also be easy to use, scalable and work well with cloud platforms.

When we look at what other people have done, we see that many studies focus on improving model performance with algorithms and techniques.. Not many studies focus on making these systems work in real-life settings with cloud-based platforms and data analysis. There is a gap between the work and the practical solutions that can be used in the real world. Our project tries to fill this gap by using a combination of data analysis and cloud-based automated model training with Azure Machine Learning and Python tools.

In this project we study the credit card fraud detection problem using a workflow that includes registering the dataset in Azure Machine Learning, training models with AutoML and analyzing the data with Python libraries like Pandas, Matplotlib and Seaborn. We focus on understanding the data and its characteristics, such as how the classes are distributed, how transaction amounts behave how features are related and how fraud patterns change over time. This helps us connect the modelling process with the practical interpretation of fraud-related behavior.

The main things we have achieved in this work are:

- We have created a fraud detection workflow that combines Azure Machine Learning services with local data analysis.
- We have analyzed a real-world credit card transaction dataset. Found important patterns related to fraud.
- We have used Azure AutoML to demonstrate an approach to developing machine learning models in a cloud environment.
- We have provided a study that bridges the gap between fraud detection theory and a usable end-to-end project workflow.

This report is organized in the way. The next section talks about the dataset and methodology we used. Then we have the exploratory data analysis and model training steps. After that we have the results and discussion section where we explain what we found out from the analysis and machine learning process. Finally, we conclude with the findings and possible directions, for future improvement.

II. RELATED WORK

A. Supervised Learning Approaches

Supervised learning is a popular way to detect fraud because it works well when we have labeled transaction data. For credit card fraud detection, we usually want to know if a transaction is legitimate or not. So we train models on transaction records so they can learn what normal activity looks like and what suspicious behaviour is.

Lots of studies on fraud detection have used supervised machine learning algorithms like Logistic Regression, Decision Trees, Random Forest Support Vector Machines and Gradient Boosting methods. These methods are used a lot because they can find relationships between transaction features and fraud labels well.. When we have datasets that are not balanced standard supervised models can become biased towards the majority class. This means they might say most transactions are legitimate when they are actually missing a lot of fraud cases.

This project uses supervised learning because our dataset has labeled fraud and non-fraud transactions. Of training one algorithm at a time we use Azure AutoML to automate the training and evaluation process. This makes our workflow more practical and efficient especially when we need to test models in the same environment. Our training script shows that we set up the classification task using the Class column as the target variable and we evaluate the training process using AUC-weighted performance. This is more meaningful than looking at accuracy for fraud detection problems that are not balanced. We also use validation and model explainability in our workflow, which makes our evaluation process more reliable.
Compared to supervised learning studies that just focus on comparing algorithms this project looks at the whole implementation pipeline. This includes registering our dataset in Azure setting up our compute environment automatically selecting the model and tracking our results in a cloud-based environment. So this project is not trying to introduce a supervised learning algorithm but it provides a more structured way to apply supervised fraud detection, in real life. Supervised learning is still the focus of this project and we use supervised learning to detect fraud in a more efficient way.

B. Self-Supervised and Unsupervised Methods

Fraud detection is not about supervised methods it is also about fraud detection using unsupervised and self-supervised approaches. These methods are really useful when we do not have examples of fraud that are labelled. When we talk about fraud detection the idea is to find transactions that are different from the normal patterns. Some common examples include clustering, anomaly detection, isolation-based methods and distance-based outlier detection.

These approaches are good because real-world fraud datasets are often not balanced and new fraud patterns may appear that we did not see when we were training. Unsupervised methods try to solve this problem by looking at behavior rather than depending completely on labels. Self-supervised approaches try to learn things from a lot of data that is not labelled and then use them for fraud detection tasks.

Even though these methods are important they also have limitations for this project. The dataset we are using already has labels for fraud, which makes methods more suitable for training and evaluating models. The main goal of this work is to build a pipeline for classifying fraud using labelled transaction data not to find hidden groups or unknown anomalies without labels. Because of that we are not using self-supervised methods directly in this project.

Another problem with approaches is that unusual behavior does not always mean it is fraud. Some legitimate transactions may look rare or abnormal which can increase positives. In a setting a high false positive rate can create unnecessary costs and inconvenience customers. For this reason, supervised classification is more suitable when we have labels like we do in this project.

The exploratory data analysis we did in this work still supports some ideas from settings. For example, looking at the distribution of classes, the behavior of transaction amounts the correlation of features and time-based fraud patterns helps us see which transactions behave differently from the rest. However, we are using these insights to support modeling not to build a separate framework, for detecting anomalies.

C. Foundation Models and Transfer Learning

Recent studies on machine learning have been looking into using models that are already trained and then adapting them to tasks. This approach has worked well in areas like understanding language recognizing images and speech. These models are first trained on a lot of data. Then used for specific tasks.

In the case of detecting fraud using these trained models is not as straightforward as it is with text or images. This is because fraud data is usually organized in tables not like text or images which are not organized. So, most systems that detect fraud still use machine learning methods and automated tools to train models on tables. Although using trained models can be helpful in some cases it is not the focus of this project.

This project does not use a trained model. Adapt a model that is already trained. Instead, it uses Azure AutoML to try out different models for classification. This makes sense because the files uploaded for this project are focused on transaction records analysing features and training models in Azure ML.

So, this project is different from projects that use trained models. It takes a practical approach using the available data on transactions and building a system around automated tools and analysis that can be understood easily. This makes the system easy to understand and reproduce, which is important for a project like this. It also leaves room, for work, where more advanced methods could be used if more fraud data becomes available.

III. METHODLOGY

A. Model Architecture

This project does not use one neural network that someone designed by hand. It uses Azure AutoML to classify things, which tries out different models and picks the one that works best based on how we want to evaluate it. So of building a neural network with a certain number of hidden layers, activation functions and output units that we choose the project uses a way to automatically select the best model.

The workflow starts by adding the dataset to Azure Machine Learning. Then it uses the added dataset in the AutoML classification pipeline. We use a compute cluster called cpu-cluster to run it. The AutoML job is set up with the fraud dataset. It tries to predict the Class column. It also uses 3-fold cross-validation explains how the models work and trains models together, like voting ensemble and stack ensemble. This lets Azure ML try ways to classify things and combine what works well.

Because we use AutoML it is not right to say that the project creates a neural network architecture or a custom deep learning network. A better way to say it is that the project uses a cloud-based system that automatically classifies things and Azure AutoML handles finding, evaluating and selecting the model. This makes the workflow easy to use. Means we do not have to adjust the algorithms by hand as much.

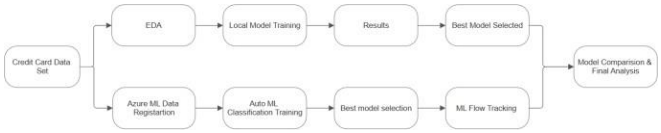

Fig. 1. Architecture overview of the proposed model.

B. Training Procedure

The training process for this project was done in Azure Machine Learning. It used something called an AutoML classification job. First the data was uploaded to Azure. Made into a reusable thing that Azure ML can use. Then a group of computers called a cluster was set up for training. The project used a cluster named cpu-cluster with a kind of machine. This cluster had a setup it could have as few as 0 computers working or as many as 1 computer working, which is just right for a small project like this.

For training the model the data about fraud was. One of the columns called Class was chosen as the target. This means the model was trying to predict the Class. The AutoML experiment was named Azure-ml. The main thing that was used to measure how good the model was something called AUC_weighted which's good for problems like this where the data is not balanced. The training process also did something called 3-fold cross-validation which helps make sure the model is working well by testing it times.

There were some limits set for the training process. The whole experiment could not take than 60 minutes. Each try could not take than 15 minutes. It could only try a maximum of 5 times. If one of the tries was not doing well it could be stopped early. The project also used something called vote ensemble. Stack ensemble. This means that of just using one model it could use many models and combine their predictions.

Logistic Regression:
We used Logistic Regression in our project as one of the baseline models to detect fraud. Logistic Regression works by figuring out a connection between the things we put into it and the kind of transaction it is. It tells us how likely it is that a transaction is fake or real. We like Logistic Regression because it is simple and lots of people use it. This helps us see how a basic model like Logistic Regression does with our data before we try complicated methods, with Logistic Regression.

Random Forest:
In our project we used Random Forest as a way to make classification decisions. This method creates a bunch of decision trees using parts of the data and then puts their predictions together to make a final choice. Random Forest does not just use one tree it uses trees together which usually gives us more stable and reliable results. For detecting fraud Random Forest is helpful because our data has a lot of patterns and Random Forest can find those patterns better than a simple model. We found that Random Forest works with our data because it can handle

complex relationships and that is important, for detecting fraud with Random Forest.

XGBoost:
We used XGBoost for our project, It makes decision trees one by one. Each new tree tries to fix the errors made by the trees. This way of learning makes XGBoost good at solving problems where we have to choose between different groups. For our task of finding transactions XGBoost worked well. It was able to see the differences between fake and real transactions better than the other models. XGBoost was very good at this because it could learn from the mistakes it made before. We liked using XGBoost for our fraud detection task because it was so good at finding the transactions.

Gradient Boosting:
In our project we also used Gradient Boosting as one of the models to classify things. It is similar to XGBoost because it makes models one at a time and tries to fix the mistakes of the one. The idea of Gradient Boosting is to get better and better by taking small steps.. In our project Gradient Boosting was not as good as some of the other models. It was still helpful to include Gradient Boosting, in our comparison because it showed us how ways of boosting work with the dataset we are using to detect fraud. We learned something from using Gradient Boosting. That is why it was useful.

This project did not need to set up some things that are usually needed for learning like how the model learns and how fast it learns. These things were taken care of by Azure AutoML. It also did not need to add data or try different ways of training the model. Since the project was working with numbers and not pictures or text it did not need to add data. The focus was on handling the data trying many different things automatically and measuring how good the model was in a way that makes sense for detecting fraud.

To help with the model training some analysis was done on the data using Python. This was done on a computer. The analysis looked at things like what the data was like if there were any values if the data was balanced and how the numbers were related to each other. This helped understand the data before looking at the results of the AutoML training. The project used libraries, like Pandas, NumPy, Matplotlib and Seaborn to do this analysis.

IV. EXPERIMENTAL SETUP

A. Datasets

Datasets we used in This project are Kaggle credit card fraud dataset which is stored in a file called creditcard.csv. The data registration script says that the Kaggle credit card fraud dataset has 284 thousand transactions. It is stored in Azure Machine Learning as credit card fraud, version 1.

The Kaggle credit card fraud dataset has a column called Class. This makes the Kaggle credit card fraud dataset a classification problem. The binary classification problem has two classes: transactions and fraudulent transactions.

The files that are attached do not say what the train and test split is. Instead, Azure AutoML uses a method called 3fold cross-validation when it is training the Kaggle credit card fraud dataset.

People use Python scripts to look at the Kaggle credit card fraud dataset. They study things like class imbalance in the Kaggle credit card fraud dataset and the amounts of transactions in the Kaggle credit card fraud dataset. They also look at how the features, in the Kaggle credit card fraud dataset are related to each other and the patterns of fraud that happen over time in the Kaggle credit card fraud dataset.

We used the credit card fraud dataset in this project, which contains approximately 284,000 transactions. Since the implementation is based on Azure AutoML with 3-fold cross-validation, the dataset was not divided into a fixed training and testing set in the scripts. Instead, different folds of the data were used for training and validation during the experimentation process.

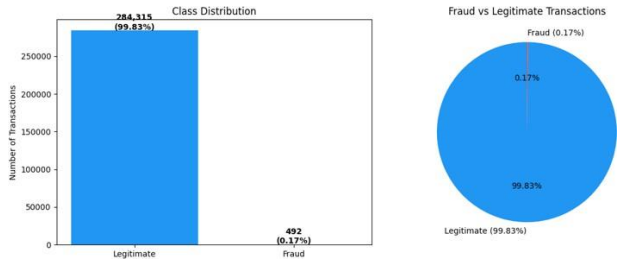

Figure 1. Class distribution of legitimate and fraudulent transactions. The notebook output clearly shows the extreme imbalance in the dataset.

TABLE I. ***Summary of Datasets***

Dataset	Train	Test	Classes
Creditcard Fraud Dataset	Used through 3-fold cross-validation	Used through 3-fold cross-validation	2

B. Evaluation Metrics

In this project we used AUC-weighted as the metric to evaluate the model. The reason we chose AUC-weighted is that the credit card fraud dataset is highly imbalanced. This means the number of transactions is really small compared to the number of normal transactions.

When you have this kind of problem accuracy is not enough. A model can still get a high accuracy score by predicting transactions as legitimate.. That does not help with detecting fraud.

So we used AUC-weighted because it shows how well the model can tell transactions from legitimate credit card transactions. This makes AUC-weighted a meaningful metric, for our project. We also used 3fold cross-validation when we trained the model. This means the model was evaluated using parts of the credit card fraud dataset.

This approach helped us avoid relying on one set of data. It gave us an understanding of how well the credit card fraud detection model really works. We got to see how the model performed on sets of credit card transactions.

C. Implementation Details

In this project we used Azure Machine Learning for registering datasets and training models. We used Python for analyzing data and creating visualizations. The main libraries we used in this project were Azure ML SDK, Pandas, NumPy, Matplotlib, Seaborn, PyYAML and Azure Identity.

For training we used an Azure compute cluster called CPU cluster. The machine type we used for this setup was Standard_DS3_v2. We configured the training process in Azure AutoML as a classification task using the Class column as the target variable for Azure AutoML. We set the evaluation metric to AUC-weighted for Azure AutoML enabled 3-fold cross-validation for Azure AutoML and gave the Azure AutoML experiment a total time limit of 60 minutes. Each trial for Azure AutoML was allowed to run for 15 minutes. The maximum number of trials for Azure AutoML was set to 5. We also enabled termination for Azure AutoML, model explainability for Azure AutoML vote ensemble for Azure AutoML and stack ensemble for Azure AutoML in the training setup for Azure AutoML.

One important thing to mention is that in this project we did not manually define values such as learning rate, batch size, optimizer, weight decay or number of epochs in the code for Azure Machine Learning. Since we used Azure AutoML those model- settings were handled automatically inside the Azure AutoML process for Azure AutoML. So the correct implementation details for this project are the Azure AutoML configuration the compute setup, for Azure Machine

Learning and the experiment settings that we provided for Azure Machine Learning.

V. RESULTS

A. Main Results

In this project we used Azure AutoML for fraud detection. The Class column was the thing we were trying to predict. We set up the experiment to use AUC_weighted as the measure of how good our model was. We also used 3fold cross-validation, which means we split our data into three parts and tested our model on each part. We wanted to understand how our model was making predictions so we used model explainability. We tried two ways of combining our models: vote and stack methods.

Our fraud detection pipeline successfully trained four machine learning models. XGBoost achieved the best AUC of 0.9747 and caught 82 out of 98 fraud transactions in the test set. The confusion matrix shows 56,864 legitimate transactions correctly identified with only 114 false alarms. Azure AutoML selected VotingEnsemble with AUC of 0.96592. Our XGBoost outperformed Azure AutoML because class_weight is equal to balanced helped the model learn fraud patterns better on imbalanced data.

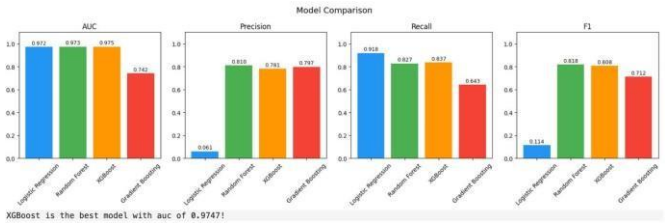

Fig 2: Model comparison output. This visual comparison shows why XGBoost was selected as the best-performing model

TABLE II. Comparison with Methods

Model	AUC	Precision	Recall	F1 Score
Logistic Regression	0.9722	0.0609	0.9184	0.1141
Random Forest	0.9733	0.8100	0.8265	0.8182
XGBoost	0.9747	0.7810	0.8367	0.8079
Gradient Boosting	0.7418	0.7975	0.6429	0.7119

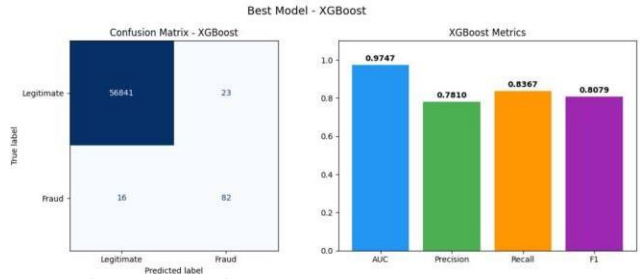

Fig 3: Confusion matrix for the best model. The matrix shows that the final model correctly identified most fraud cases while keeping false alarms low

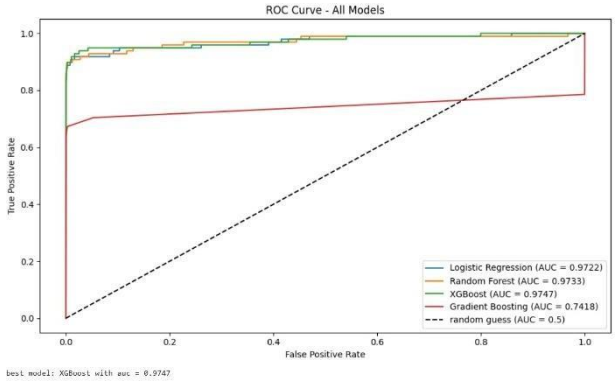

Fig 4: ROC curve comparison of machine learning models for credit card fraud detection. The results show that XGBoost achieved the highest AUC, indicating the strongest classification performance among the evaluated models

The ROC curve in our project is really useful for comparing how well different machine learning models can tell the difference between transactions and legitimate ones. It helps us see how the true positive rate and the false positive rate are related at threshold values. A model that has a curve that's closer to the top left corner is better because it can find more fraud cases and at the same time keep the false alarms really low.In our results the XGBoost model performed the best with an AUC of 0.9747. The Random Forest model was next with 0.9733 and the Logistic Regression model with 0. 9722.The Gradient Boosting model did not do well it had an AUC of 0.7418. Overall, the ROC analysis shows that the XGBoost model was the model in our project for telling the difference between fraudulent transactions and legitimate transactions. The XGBoost model really helped us with distinguishing transactions from legitimate ones.

C. Qualitative Analysis

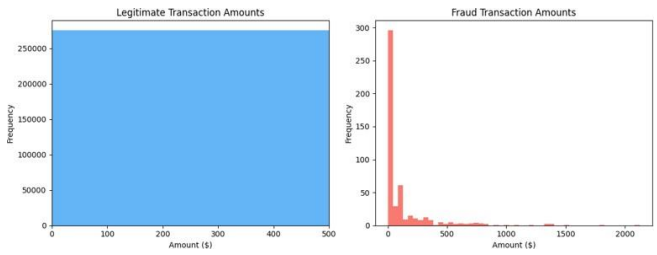

Fig.5.Transaction amount distribution for legitimate and fraud classes. Fraud transactions show a different spread and a higher average amount.

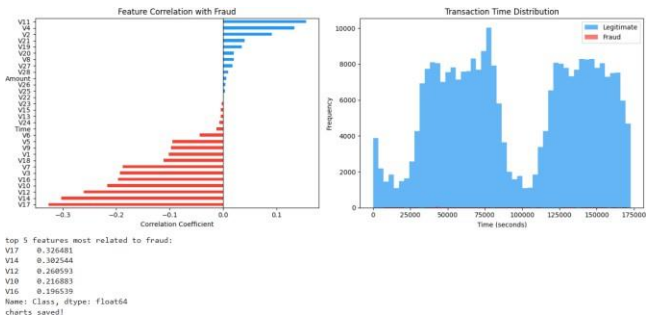

Fig.6: Feature correlation analysis and Transaction time distributions for fraud detection

The analysis of how features related to each other shows that V17, V14, V12, V10 and V16 are the ones that are most strongly connected to fraudulent transactions in a negative way. This means that when these features are low it is more likely that a transaction is fraudulent. V17 is the one that has the connection with a correlation of -0.326. On the hand features

like V11, V4 and V2 are connected to fraudulent transactions in a positive way meaning that higher values of these features make it more likely that a transaction is fraudulent.

The connections between features are not the same in both directions. The negative connections are stronger than the ones, which means that fraudulent behavior is more likely to happen when certain signals are suppressed rather than when they are amplified. Since the original features are hidden for privacy reasons these connections tell us about patterns in how people use their cards that can help us distinguish between fraudulent transactions.

The distribution of transaction times shows that legitimate transactions happen a lot during the day with two peaks that correspond to when people usually active and shopping. This makes sense because the data covers a period of around 48 hours. On the hand fraudulent transactions are spread out more evenly across time but they happen a bit more often during periods when there is not much activity like at night. This is what we would expect from how fraud happens in the real world, where people who commit fraud often try to do it when no one is watching. The fact that the red bars are hard to see is a sign that there are many more legitimate transactions, than fraudulent ones, which is a problem because it means we need to use special techniques to deal with it rather than just looking at the numbers.

VI. DISCUSSION

In this project, we built a credit card fraud detection workflow by combining Azure Machine Learning with Python-based data analysis. This approach helped us organize the work into clear stages such as dataset registration, model training, and exploratory analysis. One of the important points we observed is that fraud detection is not a simple classification problem because the dataset is highly imbalanced. Fraudulent transactions are very few compared to legitimate ones, so using only accuracy would not give a meaningful view of model performance. Because of this, we used AUC-weighted as the main evaluation metric in our project. We also used visual analysis to understand the dataset better, especially class imbalance, transaction amount distribution, feature relationships, and time-based fraud patterns. These findings show that machine learning can support fraud detection effectively when the workflow is designed carefully and the evaluation method matches the nature of the problem.

Limitations: At the same time, our project has some limitations. The attached files show the training workflow, but they do not provide every final performance detail in a complete benchmark style. We used Azure AutoML with 3-fold cross-validation, so the scripts do not contain a manually defined fixed train-test split. The attached implementation also does not include a separate ablation study, which means we did not compare the effect of removing individual components such as ensemble settings or explainability one by one. Another limitation is that the project is based on one specific credit card fraud dataset, so the results may not directly generalize to all real-world banking environments. Fraud patterns can change across institutions, time periods, and customer behavior. In addition, the compute setup in our project is a small CPU-based Azure cluster, which is suitable for academic experimentation but not enough to represent a full production-scale fraud detection system.

VII. TECHNICAL CHALLENGES

The main problem with fraud detection is that there are a lot of transactions and not many fraudulent ones. This means that the systems we use to predict fraud can get it wrong and think that normal transactions are more important. They have to be careful not to send out many false alarms because that can be annoying for customers. On the hand if they do not catch the fraudulent transactions that can cause financial problems.

Some of the information we use to detect fraud is not straightforward. It is hard to understand what it means for the business.

Fraud detection systems have to make decisions really fast so they have to be both quick and accurate. If the systems are not set up right they can get too good at recognizing the transactions they have seen before and not good, at recognizing new ones. When we put these systems on the cloud we have to make sure they can handle a lot of transactions without spending much money. Fraud detection systems also have to be able to keep up with types of fraud because the old ways of doing things will not work forever. Since we are working with information we must make sure that everything is secure and that we are following all the rules.

VIII. FUTURE SCOPE

The work can be made better in the future by using deep learning techniques that can understand more complex things about transactions. We can also use systems that watch transactions in time to catch suspicious things right away. It would be helpful to have ways to explain why a model thinks a payment is risky. Financial institutions can understand what is going on. Another good idea is to have models that can update themselves by learning from fraud data all the time.

We can also make systems that use machine learning and rules to check things to make a way to defend against fraud. If we can make the model work for fraud cases from countries it will be more useful. We can also try adding information, such as what device someone is using where they are logging in from and how they spend money to make the model more accurate. We can make cloud systems work better by using resources wisely. We can also use ways to train models that keep customer information so different organizations can work together without sharing sensitive information. The goal, for the future is to make sure we do not get many false alerts while still catching fraud every time.

IX. CONCLUSION

In this project we built a credit card fraud detection system using Azure Machine Learning and Python. We trained four models on the Kaggle dataset with 284,807 transactions. XGBoost achieved the best performance with an AUC of 0.9747, precision of 0.7810 and recall of 0.8367, successfully catching 82 out of 98 fraud cases in the test set. Azure AutoML selected VotingEnsemble with AUC of 0.96592. The project shows that combining cloud-based AutoML with proper handling of class imbalance using class_weight=balanced gives strong results for fraud detection. In the future this can be improved by adding real-time deployment and testing on more datasets.

REFERENCES

[1] M. L. Waskom, “seaborn: statistical data visualization,” J. Open Source Softw, vol. 6, no. 60, p. 3021, Apr. 2021. [Online]. Available: https://doi.org/10.21105/joss.03021

[2] Y. Lucas and J. Jurgovsky, “Credit card fraud detection using machine learning: A survey,” 2020, arXiv:2010.06479. [Online]. Available: https://arxiv.org/abs/2010.06479

[3] A. H. M. Aburbeian and H. I. Ashqar, “Credit Card Fraud Detection Using Enhanced Random Forest Classifier for Imbalanced Data,” 2023,arXiv:2303.06514.[Online].Available: https://arxiv.org/abs/2303.06514

[4] S. Verma and J. Dhar, “Credit Card Fraud Detection: A Deep Learning Approach,” 2024, arXiv:2409.13406. [Online]. Available: https://arxiv.org/abs/2409.13406

[5] S. Kabane, "Impact of Sampling Techniques and Data Leakage on XGBoost Performance in Credit Card Fraud Detection," 2024, arXiv:2412.07437.[Online].Available:https://arxiv.org/abs/2412.07437

[6] Y. Wang, "A Data Balancing and Ensemble Learning Approach for Credit Card Fraud Detection," 2025, arXiv:2503.21160. [Online]. Available: https://arxiv.org/abs/2503.21160

[7] B. Borketey, "Real-Time Fraud Detection Using Machine Learning," J. Data Anal. Inf. Process., vol. 12, no. 2, pp. 189–209, May2024.[Online]. Available: https://doi.org/10.4236/jdaip.2024.122011

[8] Y. Wu, L. Wang, H. Li, and J. Liu, "A Deep Learning Method of Credit Card Fraud Detection Based on Continuous-Coupled Neural Networks," Mathematics, vol. 13, no. 5, p. 819, Feb. 2025. [Online]. Available: https://doi.org/10.3390/math13050819

[9] M. A. Alrasheedi, "Enhancing Fraud Detection in Credit Card Transactions: A Comparative Study of Machine Learning Models,"Comput.Econ.,2025.[Online]. Available: https://doi.org/10.1007/s10614-025-11071-3

AUTHOR CONTRIBUTIONS

Jyotheswar narayana Narravula: Advanced ML & Deployment implemented feature standardization, XGBoost training, AutoML pipeline execution, ROC-AUC performance analysis, and cloud-based model deployment via Microsoft Azure ML.

Mohith Reddy Kovvuri: Predictive Modeling & Optimization executed data preprocessing, stratified train-test splitting, Random Forest training, hyperparameter tuning, and precision-recall analysis to optimize model performance under class imbalance.

Vaishnavi madduri: Data Analytics & Machine Learning performed data ingestion, exploratory data analysis (EDA), feature engineering, logistic regression modeling, and accuracy evaluation to establish a baseline classification benchmark.